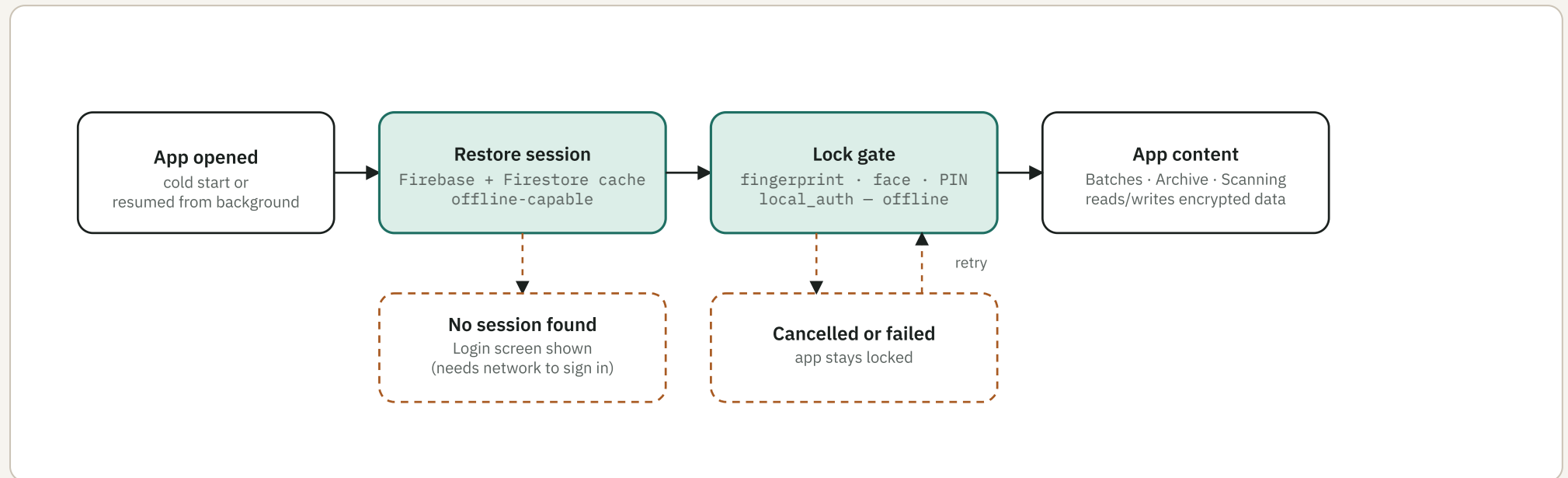


How the app keeps examinee data secure

Two independent layers, both fully offline-capable: a device fingerprint/PIN gate on every open, and `BatchCryptoService` encrypting every batch at rest with AES-256-GCM — a key held in the platform's own secure hardware, not in a file next to the data it protects.

Getting into the app

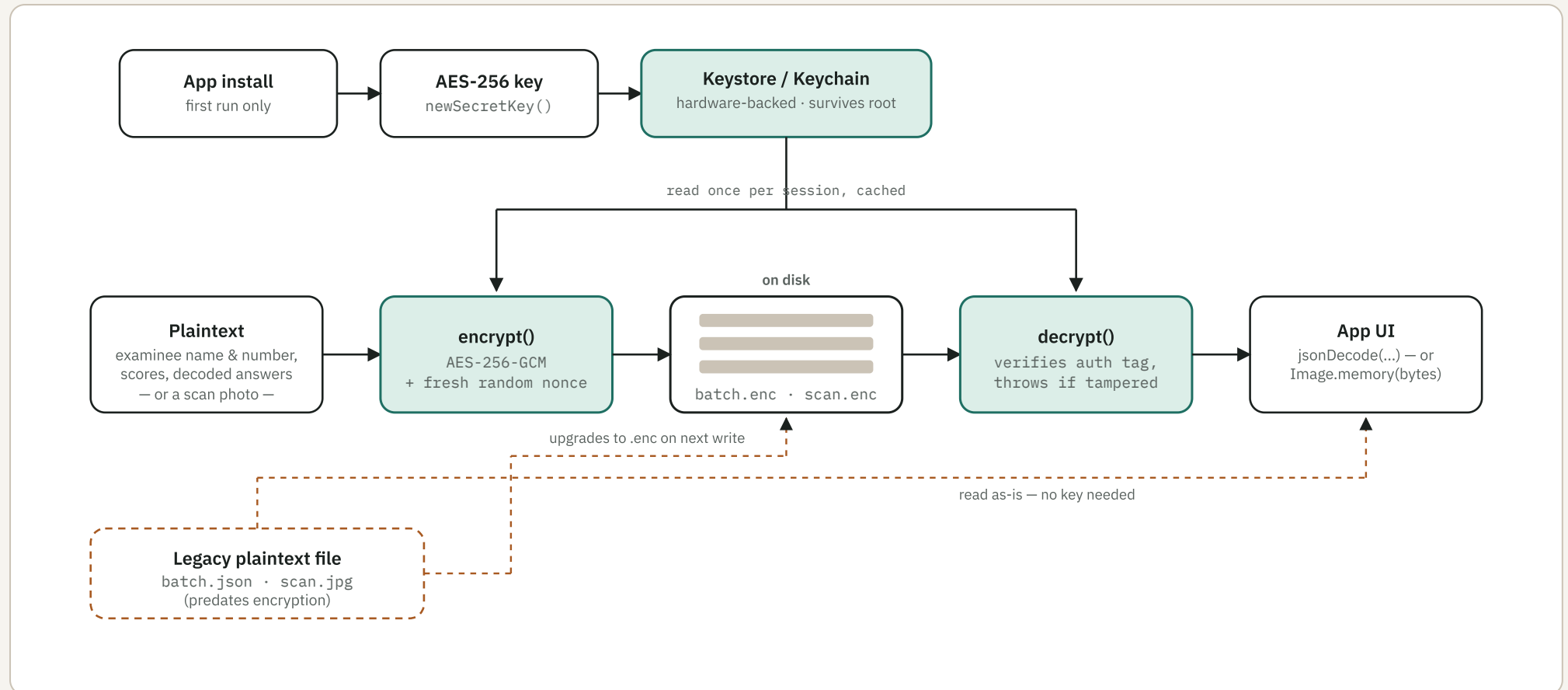
Every cold start or return from the background, before any screen renders.



Both steps run without a network connection, as long as you've signed in successfully at least once before on this device. Neither one is a substitute for the other: restoring the session answers “is this account still approved,” the lock gate answers “is the person holding this phone right now the one who signed in” — a still-approved session that nobody re-verifies locally is exactly how an unattended, unlocked phone would otherwise hand over everything in it.

Every file, at rest

What actually happens to a batch's data and photos on disk, independent of whether the app is open.



One AES-256 key, generated on first run and kept in the OS's own secure key store, is reused for every batch: the write path encrypts a fresh nonce in with each file, the read path verifies and decrypts with the same key. A batch saved before this existed keeps working unencrypted until it's next written to — a tag, a rescan, a new scan — at which point it's rewritten encrypted, same as any new batch.

PROTECTS AGAINST

- Someone reading the raw files directly — a rooted device, an ADB pull, physical extraction. Without the Keystore-backed key, the bytes are unreadable ciphertext.
- An unattended phone that's still signed in — the lock gate re-checks the person holding it, every open, closing what used to be an open gap.

DOES NOT PROTECT AGAINST

- The very first sign-in on a device, which still needs a real network connection to authenticate.
- The Supabase cloud-sync upload — a separate, already-secured plaintext-over-TLS channel.
- The device's own lock method being defeated — the app trusts whatever fingerprint/PIN the phone itself already trusts.

EXACTLY HOW

Manifest	<code>guidegrade_batches/<batchId>/batch.enc</code>
Original photo	<code>guidegrade_batches/<batchId>/images/<scanId>.enc</code>
Rectified photo	<code>guidegrade_batches/<batchId>/images/<scanId>_rectified.enc</code>
Cipher	AES-256-GCM — authenticated, a tampered file fails to decrypt rather than loading silently
Key storage	<code>flutter_secure_storage</code> , key id <code>guidegrade_batch_encryption_key_v1</code>

Package	<code>local_auth</code>
Trigger	every <code>AppLifecycleState.resumed</code> while signed in, plus cold start
Fallback	<code>biometricOnly: false</code> — falls back to device PIN/pattern/password
Host activity	<code>FlutterFragmentActivity</code>
No lock configured	skipped entirely, rather than locking you out with no way in